2. Tools and Working Environment

Data Science with Applications in Natural Language Processing

Learning goals

By the end of this chapter we will be able to:

- **explain** why Python dominates data science
- **choose** a notebook or an IDE depending on the goal
- **write** notebooks that others can read and reproduce
- **use** NumPy, Pandas and Matplotlib together
- **load** and **explore** a real dataset

Chapter map

1. Python for data analysis
2. Notebooks, IDEs and good practice
3. NumPy, Pandas, Matplotlib
4. Real-world datasets

The dataset that will follow us

Sentiment in the Iliad

611 verses · annotators chose positive, negative, neutral or “narrator”

```
text      "Τη μάνητα, θεά, τραγούδα μας του ξακουστού Αχιλλέα,"
neutral   0.375
positive  0.5
negative  0.125
narrator  0.0
```

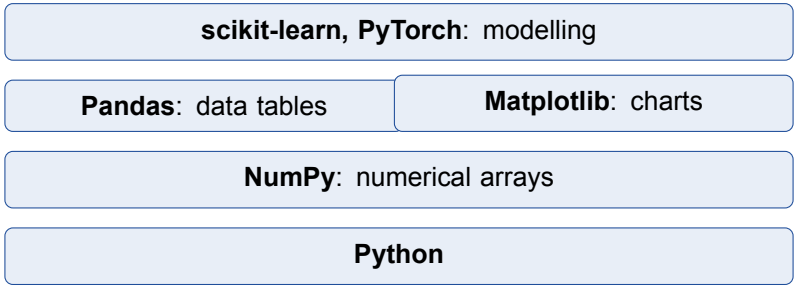
“Sing, goddess, the wrath of famous Achilles”. We load and explore it at the end of the chapter.

Why Python?

- **Low barrier to entry**: syntax close to natural language
- **Ecosystem**: libraries that work well together
- **A bridge language**: from experiment to production
- **Balance**: flexibility, with fast C/C++ code underneath
- **Community**: documentation, examples, open source

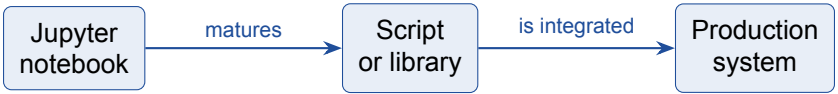
It was not designed for statistics; its ecosystem made it the de facto standard.

The ecosystem



Each layer **builds** on the one below

One language from experiment to production



- The same code, with no change of language
- Narrows the gap between research and application

Is Python slow?

Yes, as pure Python. No, in practice.

- Heavy computation runs in C/C++ (NumPy, Pandas)
- Start simple
- Optimise only the real **bottleneck**

Readability

```
sales = [187, 200, 300, 192, 201]
total_sales = sum(sales)
average_sales = total_sales / len(sales)   # 216.0
```

- Reads almost like a formula
- No loops, no helper variables
- The **intent** is clear even without comments

Dynamic typing

```
value = 42
value = "unknown"
value = [1, 2, 3]
```

Benefit
fast exploration while we do not yet know the structure of the data

Cost
type errors surface late; mature code needs care

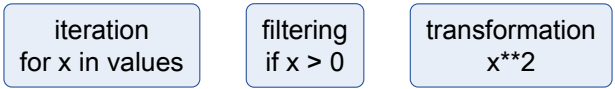
Built-in data structures

```
student = {
    "name": "Maria",
    "grades": [8, 9, 10],
    "passed": True,
}
```

- One dictionary: text, a list and a boolean together
- A natural representation of a **record**
- Usually the step before a data table

List comprehensions

```
values = [1, 2, 3]
squared_values = [x**2 for x in values if x > 0]   # [1, 4, 9]
```



Three ideas in one line: less code, fewer bugs.

Interactivity

```
values[:2]    # [1, 2]
```

- Run a little code and see the result at once
- Test every assumption with simple commands
- Step by step, instead of one large script

From scratch code to a function

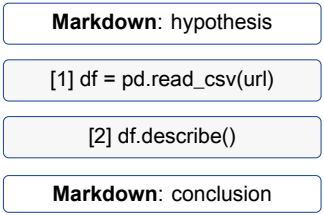
```
def normalize(values):
    mean = sum(values) / len(values)
    return [(x - mean) for x in values]

normalize([1, 2, 3])   # [-1.0, 0.0, 1.0]
```

- Reuse instead of copy-paste
- A step before the ready-made NumPy and Pandas functions

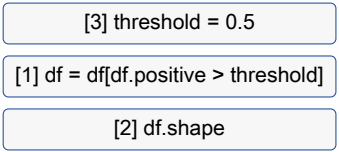
Jupyter Notebooks

- Code, text, maths and charts together
- Execution in **cells**
- The analysis becomes a **narrative**



Ideal for teaching, research and early-stage analysis.

The risk: hidden state



- The cells ran **out of order**
- From the top, cell [1] fails
- Or, worse, it runs with an old value

Rule: before you hand it in, “Restart & Run All”.

Integrated development environments (IDEs)

What they offer

- autocompletion
- error checking, debugging
- Git integration

When you need them

- the code grows
- several people contribute
- the project goes to production

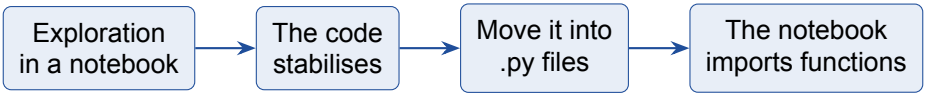
e.g. **VSCode**, **PyCharm**

Notebook or IDE?

	Notebook	**IDE**
Goal	exploration, communication	structured, reliable code
Flow	non-linear, by cell	linear, by file
Strength	instant view of the data	maintenance, collaboration
Risk	hidden state	slower experimentation

They do not compete; they **complement** each other

From notebook to IDE: a sign of maturity



The move is not a change of tool but a **maturing of the analysis**

Good practice, whatever the tool

- **Clear names** for variables
- **No repetition**: functions instead of copy-paste
- **Separation**: loading · processing · visualisation/model
- **Git**, even for solo projects

Git lets you go back to an earlier state without losing work.

Bad notebook: a cell with no context

```
df = df.dropna()
```

- Where did df come from?
- What data does it hold?
- **Why** drop missing values? From which columns?

Run from the top, the cell fails or relies on hidden state.

Bad notebook: meaningless names and charts

```
X = data[:, 1:]
y = data[:, 0]
```

Which is the target? Which are the features? What is data?

```
plt.plot(x, y)
```

No title, labels or comment: the chart “tells no story”.

Good notebook

```
# Data load from CSV file
df = pd.read_csv("sales.csv")

# Remove instances with missing values
df = df.dropna(subset=["price", "quantity"])

# Separate the target variable from the rest
X = df[["price", "discount"]]
y = df["quantity"]

plt.plot(df["price"], df["quantity"])
plt.xlabel("Product price")
plt.ylabel("Units sold")
plt.title("Price and demand")
```

Bad and good notebooks

Bad

- code with no narrative
- depends on cell order
- no documentation
- results with no interpretation

Good

- clean, linear flow
- code that stands on its own
- comments on the **why**, not the how
- a visible link between code and conclusions

A notebook is judged by whether it reads and reproduces, not by whether it “runs”.

Checklist before you hand in

Structure

- runs top to bottom with no errors
- an opening cell states the goal

Code

- descriptive names
- no “magic” variables

Documentation

- comments on key decisions
- conclusions in words

Charts

- titles and axis labels
- clear which question each answers

Reproducibility

- imports at the top
- data with an explained source
- someone else can repeat it

Exercise: what would you fix?

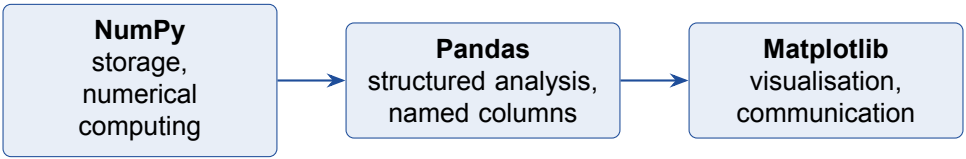
```
[4] import matplotlib.pyplot as plt
[2] d = pd.read_csv("/Users/maria/Desktop/final_v3.csv")
[5] d2 = d[d.c3 > 0.5]
[3] import pandas as pd
[6] plt.hist(d2.c3)
```

Find at least five problems using the checklist.

Exercise: answers

- Cells **out of order**: pd is used before it is imported
- **Imports** not at the top
- A **local path** with no explanation: it will not run elsewhere
- Meaningless **names**: d, d2, c3
- A **magic** threshold of 0.5 with no justification
- A **chart** with no title, labels or interpretation

Three layers, one workflow



- Designed to **work together**
- Shared conventions → collaboration and reproducibility

The foundation of almost the whole ecosystem

Unlike Python lists:

- **homogeneous type**: all integers or all floats
- **memory-efficient** (contiguous storage)
- **bulk operations** with no explicit loops

Creating an array

```
import numpy as np

data = np.array([1, 2, 3, 4, 5])
data.shape    # (5,)
data.dtype    # dtype('int64')
```

- shape: the dimensions of the array
- dtype: the type of its elements
- Critical once data get large

Vectorisation

Python loop

```
result = []
for x in data:
    result.append(x * 2)
```

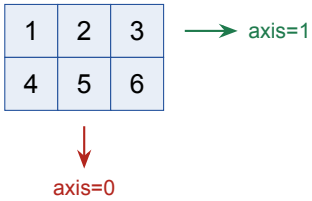
Bulk operation

```
data * 2
# array([ 2, 4, 6, 8, 10])
```

- Shorter, clearer
- Runs in C → **much faster**

Shape and axes

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
matrix.shape        # (2, 3)
matrix.sum(axis=0)  # [5, 7, 9]
matrix.sum(axis=1)  # [6, 15]
```



The same axis logic applies in Pandas.

Statistical functions

```
values = np.array([10, 12, 9, 15, 11])
values.mean()  # 11.4
values.std()   # 2.059
values.min()   # 9
values.max()   # 15
```

- Optimised for speed
- The first step before more complex techniques

When do we use NumPy directly?

- Intensive numerical computation
- Data **without names** (columns, indices)
- Full control over shape and structure
- Implementing or understanding algorithms at a low level

We rarely work with NumPy alone, but it sits behind almost every tool.

Pandas: Series and DataFrame

Series

```
import pandas as pd
ages = pd.Series(
    [23, 45, 31, 29])
```

one-dimensional, indexed, allows missing values

DataFrame

```
df = pd.DataFrame({
  "name": ["Anna", "Nikos", "Maria"],
  "age":  [28, 34, 25],
  "city": ["Athens", "Thessaloniki",
           "Patras"]})
```

two-dimensional, with named columns

A first look in three commands

```
df.head()      # the first rows
df.info()      # types and missing values
df.describe()  # statistics for numeric columns
```

- head: visual inspection
- info: what type is each column? is anything missing?
- describe: distributions at a glance

Almost always run before any model.

Selection and filtering

```
df["age"]              # one column
df[df["age"] > 30]     # only the rows that meet the condition
```

```
   name  age         city
1 Nikos   34 Thessaloniki
```

- Close to natural language
- Replaces loops, reduces errors

Missing values

```
df.isna()      # where are values missing?
df.dropna()    # drop rows
df.fillna(0)   # replace
```

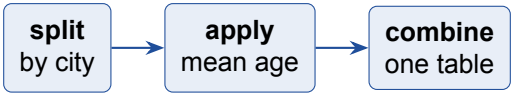
Every choice is a decision

fillna(0) on an age says “this student is 0 years old”. dropna() may systematically remove a whole group.

More in chapter 4.

Grouping: split–apply–combine

```
df.groupby("city")["age"].mean()
```



city	age
Athens	28.0
Patras	25.0
Thessaloniki	34.0

Pandas on top of NumPy

A more “human” face, the same engine

- Inside: NumPy arrays
- It inherits their efficiency
- Convert DataFrame ↔ array whenever needed

Pandas: cleaning, exploring, transforming, summarising.

Matplotlib: data → chart

```
import matplotlib.pyplot as plt

values = [3, 7, 5, 9, 4]
plt.plot(values)
plt.show()
```

- The position of each value becomes the x axis
- Full control: from quick exploration to publication
- The base for Seaborn, Pandas plotting and others

Titles and labels

```
plt.plot(values)
plt.title("Values over time")
plt.xlabel("Time")
plt.ylabel("Value")
plt.show()
```

The most common beginner mistake:

a chart with no title and no labels

Bar chart or histogram?

Bar chart

```
plt.bar(["A", "B", "C"],
        [10, 5, 8])
```

categorical data, comparing sizes

Histogram

```
data = np.random.normal(
    0, 1, 1000)
plt.hist(data, bins=30)
```

distribution, skew, outliers

Discussion: how does the histogram change with bins=5 or bins=200?

Visualisation as a tool for thinking

A good chart

- reveals errors in the data
- raises new questions
- guides modelling

Visualisation: chapters 6 and 7.

A bad chart

- misleads
- hides critical information

One workflow: from messages to a chart

```
messages = ["I really like this product!",
            "I did not like the service at all.",
            "Good experience overall.",
            "Disappointed, it was expensive.",
            "I recommend it without reservation!"]
df = pd.DataFrame({"message": messages})
df["length"] = df["message"].apply(len)   # [27, 34, 24, 31, 35]
df.describe()                             # mean 30.2

plt.bar(df.index, df["length"])
plt.ylabel("Message length")
plt.title("Length of messages")
```

Careful: len counts characters, including spaces and punctuation, not words.

Real data are not “clean”

Noise	typos, abbreviations, emoticons
Missing values	fields never recorded, or lost
Inconsistencies	the same information, represented differently
Bias	over-representation of groups or language communities

Example: comments on a film

```
df = pd.DataFrame({
   "text": ["I loved the movie so much!!!",
            "…unacceptable wont watch it again",
            "Perfect :)",
            "dont know if i liked it",
            "PERFECT!!!",
            None],
   "sentiment": ["positive", "negative", "positive",
                 None, "POS", "negative"]})
```

How many problems can you spot before running a single command?

What we found

Noise

- “!!!”, “:)”
- CAPITALS
- missing apostrophes

Missing values

- 1 text
- 1 label

Inconsistency

- positive and POS

One df.info() shows the missing values at once.

And is “Perfect :)” positive, or sarcastic?

Data sources

Source	Pros	Watch out
Kaggle	ready to use, with notebooks and benchmarks	often already processed
Open data portals	public policy, legislation, consultations	need interpretation and cleaning
APIs	current, real-time	rate limits, schema changes, terms of use
Web scraping	when there is no API	fragile, terms of use
Research datasets	clear labels and protocols	specific assumptions, limits to generalisation

Loading data

```
url = ("https://raw.githubusercontent.com/ipavlopoulos/"
       "sentiment_in_homeric_text/main/ib1_sentiment_probs.csv")
df = pd.read_csv(url)                    # from a URL

df = pd.read_csv("data/ib1_sentiment_probs.csv",
                 encoding="utf-8", sep=",") # locally
```

- From a URL: **reproducibility**
- encoding: critical for non-English text
- The same logic for Excel (read_excel) and JSON

First contact: size

```
df.shape    # (611, 6)
```

- **611** rows: one verse of the Iliad per row
- **6** columns: the text, four probabilities, and y

The first question for any dataset: what does one row represent?

First contact: types

```
df.info()
#  Column    Non-Null  Dtype
#  text      611       object / string
#  neutral   611       float64
#  positive  611       float64
#  negative  611       float64
#  narrator  611       float64
#  y         611       object / string
```

- No missing values
- Probabilities as float64: as expected
- Text as object (or string in pandas 3)

What does each column represent?

- text: the verse
- neutral, positive, negative: share of annotators per sentiment
- narrator: the narrator speaks, not a character
- y: ?

y, after checking

"[0.375, 0.5, 0.125, 0.0]": the same four values in one list, probably as a supervision signal. But it is stored as **text**, not as a list.

First contact: statistics

	neutral	positive	negative	narrator
mean	0.17	0.39	0.36	0.08
std	0.16	0.27	0.29	0.24
min	0.00	0.00	0.00	0.00
25%	0.00	0.25	0.12	0.00
50%	0.12	0.38	0.38	0.00
75%	0.25	0.62	0.62	0.00
max	0.75	1.00	1.00	1.00

df.describe(): what do you notice before the next slide?

What the numbers tell us

- Positive (0.39) and negative (0.36) are more common than neutral (0.17)
- Median 0.38: often **no certainty**, intermediate values
- narrator: 75% are zero · the narrator rarely speaks
- Neutral: maximum 0.75 → **not a single verse** got full agreement

These are the observations we look for from the very first contact.

Change the display, not the data

```
df.describe().style.format("{:.2f}")

df.describe().style \
  .format(lambda x: f"{x:.0f}" if x == int(x) else f"{x:.2f}") \
  .background_gradient(cmap="Blues") \
  .set_caption("Descriptive statistics")
```

- The Styler changes only the **display** in the notebook
- The data themselves stay untouched

A first visualisation

```
df["positive"].hist()
plt.xlabel("Probability of positive sentiment")
plt.ylabel("Number of verses")
plt.show()
```

- Goal: the **range** and behaviour, not final conclusions
- Is the dataset heavily imbalanced?
- Do the values cluster somewhere?

Here the answer to both is “no”.

Why first contact matters

Wrong assumptions here →
“correct” models on misunderstood data

Systematic exploration is an **integral part** of the analysis,
not an optional preliminary step.

Summary

1. Python dominates through readability, ecosystem and community
2. Notebooks for exploration, IDEs for mature code
3. A notebook is judged by whether it reproduces
4. NumPy → Pandas → Matplotlib: one workflow
5. First contact: shape, info, describe, one chart, questions

Discussion questions

1. A notebook “runs” on your machine. What could break on a colleague’s?
2. The neutral column never reaches 1. A problem with the data, or information about the task?
3. A Kaggle dataset is “clean”. What might have been lost in the cleaning?